KMIP on PKCS#11

Delivery Plan — a complete KMS with a REST control plane

Twelve steps · design, implementation and test coverage for each

Baseline	e44a2ef — 816 tests green, 41 of 53 KMIP 2.1 operations
Starting position	42 of 74 assessed features are short of full coverage
This plan closes	28 of those 42, across 12 steps
Deliberately not closed	14 — supplier, procurement, external event, or out of product scope
Method	Every step: a design, an implementation, a test plan, and a gate that must be demonstrated
Date	10 September 2026

1 How this plan works

Phases 0 to 5 of this project succeeded because each had a gate — a condition someone could watch you meet — rather than a checklist. This plan keeps that structure and adds the two things the request asks for: a design deliverable before implementation, and a named test plan rather than a promise of coverage.

Each step carries five things

Part	What it means
Objective	One sentence. If it needs two, the step is two steps.
Design	Written and reviewed before code. Names the interfaces, the data, and the decisions that are hard to reverse.
Implementation	What changes, and equally what deliberately does not.
Tests	Named cases, not a coverage percentage. Every step adds regressions for the behaviour it introduces, and the existing suite must stay green.
Gate	A demonstrable condition. The step is finished when the gate is shown, not when the code is written.

Rules every step inherits

- Tests run live against a real SoftHSM2 token. The token is never mocked — the defects worth catching are in the boundary.
- The suite stays green at every step. A step that leaves it red is not finished, and the next step does not start.
- Anything that could not be tested in this environment is recorded as untested rather than shipped as though it had been.
- Documents are regenerated, not hand-edited, and the gap matrix's ASSESSED_AT is bumped whenever coverage changes.

The arithmetic in this document is checked at build time against generate_kms_gap_matrix.py. If a step claims a feature the matrix does not list as open, claims one twice, or an open feature appears in neither the plan nor the deferred list, the generator refuses to build. That is deliberate: a plan whose numbers do not reconcile with the assessment it derives from is worse than no plan.

2 What the plan closes

Of 74 assessed features, 42 are short of full coverage today — 16 partial and 26 absent. The twelve steps close 28 of them. The remaining 14 are listed in section 5 with the reason each is not engineering work this plan can schedule.

Step	Stage	Title	Features closed
1	A	Service layer and guard	—
2	A	Query and reporting layer	2
3	B	HTTP transport, sessions and service accounts	3
4	B	Read endpoints	—
5	B	Write endpoints, and the governance gaps	3
6	B	Administration, OpenAPI, separation of duties	2
7	B	Web console and self-service	2
8	C	Enterprise identity and policy	2
9	C	Audit externalisation, compliance and ceremony	5
10	D	Storage abstraction and PostgreSQL	1
11	D	High availability, failover and deployment	6
12	D	Multi-tenancy	2

The four stages

Stage	Name	Character
A	Foundation	No new attack surface. Makes the rest safe to build.
B	The REST control plane	The API, then the console that uses it.
C	Enterprise fit	What an auditor and a security team ask for.
D	Scale, availability and tenancy	Needs infrastructure to verify.

Steps 1 and 2 close nothing and are still the two most important. They are the reason the ten that follow can be built without each one re-litigating authorization, and the reason a REST endpoint cannot quietly become a path around dual control.

3 Sequence and dependencies

The order is dictated by dependency, not by visibility. Three constraints fix most of it.

- Step 1 precedes every other step. Authorization, dual control and audit currently live inside the KMIP dispatcher; a REST layer built before they move would bypass all three, and the bypass would be silent.
- Step 10 precedes step 11, which precedes step 12. SQLite has no multi-writer story, so clustering waits on PostgreSQL, and tenancy is only worth building on something that can survive a node failure.
- Step 3 precedes 4, 5 and 6, which precede 7. The console is a client of the API; building it earlier means building it twice.
- Steps 8 and 9 depend only on stage B and can run in parallel with each other, or with stage D if there are two teams.

Where the plan needs infrastructure it does not have

Step	Needs	If unavailable
10	A PostgreSQL instance in CI	The abstraction can be built and the SQLite path kept green, but the PostgreSQL implementation cannot be claimed as verified.
11	Multiple nodes and a second HSM token	Failover must not be marked done. An untested failover path invites exactly the reliance it cannot support.
11	Network access to a container registry	The image and CI workflow have still never been executed. This step is the first opportunity to fix that.
7	A browser in CI for end-to-end tests	Playwright and Chromium are available in this environment, so this one is not blocked.

4 The steps

Sizes are estimates for planning, not commitments. The test counts are the more meaningful figure: they say what the step has to prove.

Step 1 — Service layer and guard

`Stage A · ~600 lines moved, ~15 new tests. No new dependencies.`

Make authorization, dual control, audit and metrics impossible to bypass — before a second transport exists to bypass them.

Design

- A new kmip_pkcs11/service/ package: CallContext (identity, peer address, transport), a guard() context manager, and KeyService / AdminService facades over the existing handlers and store.
- The guard runs the role allowlist, then dual control, then the operation, then audit and metrics — the exact order dispatch() uses today.
- OperationDispatcher shrinks to: decode TTLV, call the service, encode TTLV.

Implementation

- Move check_operation_allowed, dual_control_enforce, _audit and _record_metric out of the dispatcher into the guard. Operation handlers are not touched.
- Every service method takes a CallContext as its first argument, so a caller cannot forget to say who is asking.

Tests

- The existing suite must pass unmodified — this step changes no behaviour, and a test that needed editing would mean it did.
- New: a reflection test that enumerates every public method on the service classes and asserts each one is guard-wrapped. This is the test that stops step 5 from quietly reintroducing a bypass.
- New: guard writes an audit row on success and on failure; consumes a dual-control approval before the handler runs, not after; records metrics on both paths.

Gate

All 816 existing tests pass with no test file modified, and the reflection test proves no service method escapes the guard.

Closes

Nothing directly — this step exists so that the steps after it can be built safely.

Step 2 — Query and reporting layer

Stage A · ~400 lines, ~20 tests.

Let the store answer the questions a console asks, in one query rather than N+1.

Design

- store.locate_page(filters, sort, cursor, limit) returning full rows, an opaque cursor and a total — not the bare identifiers Locate returns.
- Cursor is (sort key, uid) encoded, not an offset: offset paging over a table that is being written double-shows rows.
- store.inventory_report() and store.algorithm_usage() for the two reporting gaps.

Implementation

- Add the new methods beside the existing locate(), which stays exactly as it is for the KMIP handler.
- Indexes to match the sort columns the console offers.

Tests

- Paging over a table mutated between pages neither duplicates nor skips a row — insert and delete between fetches and assert the union.
- Sort by each supported column, ascending and descending; total equals a brute-force count; a tampered cursor is rejected rather than trusted.
- Inventory and algorithm counts match a brute-force scan of the same store.

Gate

A store holding 12,000 objects returns page one of fifty, sorted by expiry, in a single query, with the same figures a full scan produces.

Closes

- Key inventory and discovery
- Crypto-agility reporting

Step 3 — HTTP transport, sessions and service accounts

Stage B · ~900 lines, ~35 tests.

An authenticated HTTPS listener that shares nothing with the KMIP port, and cannot be flooded.

Design

- New api: configuration section — own port, TLS mandatory, off by default. Not the observability port: /metrics and /ready are deliberately unauthenticated and must stay that way.
- Opaque 256-bit tokens stored as SHA-256 hashes, with identity, label, created / expires / last used / revoked. Not JWTs: revocation matters more than statelessness, and there is already a database.
- Browsers get an httpOnly, Secure, SameSite cookie plus a CSRF token; machine clients send a bearer header. Long-lived labelled tokens are the service-account mechanism.
- A bounded connection semaphore and a per-identity token bucket.

Implementation

- kmip_pkcs11/api/ with a small stdlib router on ThreadingHTTPServer — the project has three runtime dependencies and none of them does cryptography, which is a property worth keeping.
- Middleware order: authenticate, rate limit, route, translate exceptions into RFC 9457 problem+json.
- POST and DELETE /api/v1/session, GET /api/v1/me.

Tests

- Login succeeds and fails correctly; a disabled identity is refused; a deleted identity's live session stops working immediately.
- Token revocation takes effect on the next request; absolute and idle expiry; a cookie request without a CSRF token is refused while a bearer request is not.
- Rate limiting returns 429 and recovers; the connection cap holds under 200 concurrent connections; the KMIP port is unaffected throughout.
- Configuration validation refuses api.enabled without TLS.
- Every login, logout, token issue and token revoke is an audit row.

Gate

Two hundred concurrent connections do not exhaust the listener, a revoked token is refused on its next use, and the KMIP port serves normally while the API is under load.

Closes

- API keys / service accounts
- Connection and rate limiting
- Request bounds and DoS resistance

Step 4 — Read endpoints

Stage B · ~700 lines, ~40 tests.

Everything a console displays, with authorization identical to KMIP.

Design

- GET /keys (paged, filtered, sorted), /keys/{uid}, /keys/{uid}/attributes, /audit, /audit/verify, /approvals, /reports/expiring, /reports/inventory.
- 404 and 403 are chosen deliberately: an object the caller may not see returns the same answer as one that does not exist, so the API does not become an existence oracle.

Implementation

- Read-only handlers calling the services from step 1 — no store access from the transport layer, which the guard reflection test enforces.

Tests

- Authorization parity: for a given identity, GET /keys returns exactly the set KMIP Locate returns. Parametrised over owner, admin, grantee, group member and stranger.
- The paging contract from step 2 holds through the HTTP layer.
- Audit filters by identity, object, result and time range; the approval queue lists pending requests only.
- A read of an object the caller may not see is indistinguishable from a read of one that does not exist.
- Every read is audited — 'who exported this key' is a read.

Gate

A read-only console renders key inventory, audit search and the approval queue against real data, and a test proves no write path is reachable through any registered route.

Closes

Nothing directly — this step exists so that the steps after it can be built safely.

Step 5 — Write endpoints, and the governance gaps

Stage B · ~1,100 lines, ~55 tests.

Lifecycle and delegation over REST, with governance applying by construction rather than by remembering.

Design

- POST /keys; POST /keys/{uid}/activate | revoke | rekey; DELETE /keys/{uid}; PUT /keys/{uid}/cryptoperiod; grants, groups and role permissions; POST /approvals/{id}/approve.
- A dual-control refusal returns 403 carrying the approval request id and the count outstanding, so the console can render 'awaiting 2 approvals' and link to the queue. That falls out of the existing enforcement.
- Three matrix gaps are closed here because they are lifecycle work: asymmetric auto-rotation, Shamir threshold split keys, and batch atomicity via a service-level transaction.

Implementation

- All writes go through the guard. Scheduled rotation extends to key pairs using the existing ReKeyKeyPair handler; CreateSplitKey gains a Shamir method alongside XOR; the service layer wraps a batch in one transaction so a mid-batch failure rolls back.

Tests

- The dual-control assertion from the KMIP wire test, run verbatim against REST: the first Destroy is refused, the key survives, two other identities approve, the retry succeeds, and the approval is spent.
- Role allowlists and group grants behave identically over both transports — parametrised over the transport, one test body.
- Asymmetric rotation cross-links both new objects to both old ones.
- Shamir: any k of n shares reconstruct, any k-1 fail.
- A batch whose third item fails leaves the first two unapplied.

Gate

The KMIP dual-control test and the REST dual-control test share one body and both pass.

Closes

- Automatic rotation
- Split knowledge / M-of-N shares
- Bulk and batch operations

Step 6 — Administration, OpenAPI, separation of duties

Stage B · ~800 lines, ~45 tests.

Complete the API surface and split the administrator so no one role can both grant access and use it.

Design

- Identity, role and group administration endpoints.
- The admin role splits into security-admin (identities, roles, policy) and key-admin (objects, cryptoperiods), so a key administrator cannot grant themselves access to what they administer.
- An OpenAPI 3.1 document generated from the router's route table, not hand-written — a hand-written spec drifts within a month.

Implementation

- Migration assigns both new roles to anyone holding admin today, so an upgrade changes nobody's access until an operator splits them.

Tests

- A key-admin cannot create an identity; a security-admin cannot read key material; neither can escalate to the other.
- The generated OpenAPI document validates against the 3.1 schema, and a reflection test asserts every registered route appears in it.
- Responses for a sample of endpoints match their declared schemas.

Gate

openapi.json validates, covers every route the router knows about, and the two admin roles are provably disjoint in capability.

Closes

- REST / JSON API
- Separation of duties

Step 7 — Web console and self-service

Stage B · ~2,000 lines of front-end, ~30 tests.

The interface most of the market considers table stakes, and the reason the API exists.

Design

- A single-page client of the API. No privileged path: the console can do nothing the API forbids, and holds no credential the API would not accept from curl.
- Views: key inventory, key detail, approval queue, audit search, expiry report, identity and role administration.
- Self-service: a team member creates and rotates keys within their group's grants, without an administrator.

Implementation

- Served by the API listener or a CDN. Strict CSP with no unsafe-inline. Session in an httpOnly cookie, never in localStorage.

Tests

- End-to-end with a headless browser (Playwright is available): log in, list keys, open the approval queue, approve a request, watch a blocked Destroy succeed on retry.
- The CSP contains no unsafe-inline and no unsafe-eval.
- A route the API forbids to this identity is not reachable from the console by any means, including direct navigation.

Gate

An operator completes a full day of routine work — provisioning, approving, investigating an audit question — without touching the CLI.

Closes

- Web console
- Self-service developer portal

Step 8 — Enterprise identity and policy

Stage C · ~900 lines, ~40 tests.

Authenticate against the corporate directory, and express conditions the role model cannot.

Design

- OIDC: validate the ID token (issuer, audience, expiry, signature) and map a claim to a provisioned identity. LDAP/AD bind as an alternative.
- The mTLS path already proves the pattern — an external assertion resolved to a provisioned principal, never inventing one.
- Attribute-based conditions evaluated in the guard before the allowlist: time window, source network, purpose. The engine may only narrow.

Implementation

- Directory group to role mapping, refreshed on login.
- Policies stored in the database, versioned, and every evaluation that denies is audited with the rule that fired.

Tests

- An expired, wrong-audience, wrong-issuer or unsigned token is refused; a valid token for an unprovisioned subject is refused.
- Directory group changes take effect on next login.
- A policy denies outside its window and permits inside it.
- Property test: for any identity and any policy set, the permitted operation set is a subset of what the role model alone would allow. A policy that widens access is a bug by construction.

Gate

A corporate login yields exactly the roles the directory says, and a policy denial names the rule that fired in the audit log.

Closes

- Enterprise IdP — LDAP/AD, SAML, OIDC
- Attribute or policy-based access

Step 9 — Audit externalisation, compliance and ceremony

Stage C · ~1,000 lines, ~45 tests.

Make the audit trail survive an attacker with file access, and produce the evidence an auditor asks for.

Design

- A shipper emitting audit rows as syslog/CEF, with a persisted cursor so a restart neither duplicates nor loses entries.
- Periodic notarisation of the chain digest to an append-only external endpoint. This is the one thing the local chain cannot do: detect a wholesale, internally consistent rewrite.
- Compliance report generators over the audit log and key inventory — PCI DSS key-management evidence, NIST SP 800-57 cryptoperiod evidence.
- A key ceremony runbook, and a kmip-admin command that produces a signed transcript of what was done, by whom, witnessed by whom.
- Alerting rules over the metrics already exported.

Implementation

- The shipper runs in worker 0 alongside the scheduler, with the same single-instance reasoning.

Tests

- The shipper emits every row exactly once across a restart — kill it mid-stream and assert the union at the collector.
- CEF output parses with a standard parser.
- Anchoring detects tampering the local chain cannot: rewrite the whole log consistently, re-chain it so verify_audit_chain passes, and assert the anchor comparison fails.
- Report figures match direct store queries.
- A ceremony transcript verifies against its signature and fails after a single byte is changed.

Gate

A rewritten but internally consistent audit log — one that passes local verification — is caught by the external anchor.

Closes

- Formal key ceremony
- External audit anchoring
- SIEM integration
- Compliance reporting

- Alerting

Step 10 — Storage abstraction and PostgreSQL

Stage D · ~1,500 lines, ~60 tests. Needs a PostgreSQL instance in CI.

Remove the single-writer constraint that blocks everything in stage D.

Design

- Extract a Store interface; SQLite becomes one implementation and PostgreSQL another.
- Each SQLite-specific mechanism needs a deliberate equivalent, not a translation: PRAGMA user_version becomes a schema-version table, json_extract becomes JSONB operators, RAISE(ABORT) triggers become PL/pgSQL triggers, and BEGIN IMMEDIATE — which is what keeps the audit chain from forking — becomes an advisory lock or SERIALIZABLE.

Implementation

- Both backends ship. SQLite stays the default for single-node deployments; it is a good answer there and removing it would be a regression.

Tests

- The entire existing store suite runs against both backends from one parametrised fixture — not a parallel copy that drifts.
- The audit chain survives concurrent writers on PostgreSQL: hammer it from several connections and assert the chain verifies and no sequence number is reused.
- Migrations apply identically on both; backup and restore work on both.
- A store dump from one backend restores into the other.

Gate

The full suite — 816 tests plus everything added since — passes against PostgreSQL as well as SQLite.

Closes

- Enterprise database backend

Step 11 — High availability, failover and deployment

Stage D · ~1,800 lines, ~55 tests. Needs multiple nodes and a second token.

Survive the loss of a node, a token, or a site.

Design

- Several nodes against one PostgreSQL. The scheduler takes a leader lock so exactly one instance enforces cryptoperiods — the same reasoning that restricts it to worker 0 today, one level up.
- An HSM pool: several tokens, health-checked, with failover. The master key must exist on each, which makes provisioning part of the design rather than an afterthought.
- Scheduled backup with offsite shipping; Helm chart and manifests; the container image and CI workflow finally executed.

Implementation

- Read traffic spreads across nodes; audited writes still serialise on the chain, so this raises availability more than throughput. Say so rather than implying linear scaling.

Tests

- Kill the leader under load: another takes over within the lease period, and no key is deactivated twice.
- Fail a token mid-operation: the request either completes on another or fails cleanly, never silently half-applies.
- Two nodes serving concurrently produce one intact audit chain.
- A scheduled backup taken on one node restores on another.
- The container image builds and its health check passes — the first time this has been executed anywhere.

Gate

A node is killed under sustained load: no request is lost, the audit chain verifies, and cryptoperiod enforcement continues uninterrupted.

Closes

- HA clustering
- Multi-site replication and DR
- HSM failover and pooling
- Horizontal throughput
- Scheduled and offsite backup
- Kubernetes / container deployment

Step 12 — Multi-tenancy

Stage D · ~1,600 lines, ~80 tests.

Let unrelated tenants share one deployment without seeing each other.

Design

- Tenant becomes a first-class column on objects, identities and audit rows, and every query is scoped by it.
- Object names become unique per tenant rather than globally — the change most likely to surprise an existing deployment.
- Quotas: object count, operations per second, storage. Per-tenant audit views. Tenant administrators who cannot see other tenants.
- This is why it is last: the boundary reaches into every query, the audit log, the CLI and the API, and half of it would be worse than none.

Implementation

- A migration places every existing object in a default tenant, so a single-tenant deployment behaves exactly as before.

Tests

- Exhaustive isolation: parametrised over all 41 KMIP operations and every REST route, an identity in tenant B cannot read, modify, locate or destroy an object in tenant A. This is the test that matters and it should be generated from the operation table, not hand-written.
- Name collisions across tenants are allowed; within a tenant they are not.
- Quota enforcement at the boundary, and the audit row that records it.
- A tenant administrator cannot escalate out of their tenant.
- Audit search returns only the caller's tenant.

Gate

A generated test over every operation and every route proves no cross-tenant reachability exists.

Closes

- Multi-tenancy and namespaces
- Per-tenant quotas

5 What the plan does not close

14 features remain open after step 12. None is an oversight; each is here because it depends on something other than engineering time, or because it belongs to a different product.

Feature	Why not now	Detail
Cloud BYOK (AWS, Azure, GCP)	Additive work, best done after stage B	Per-cloud connectors calling each provider's import API. Well understood, additive, and best done once the REST layer exists to configure it.
Cloud EKM / XKS / hold-your-own-key	Additive work, best done after stage B	Implement the AWS XKS proxy or GCP EKM API in front of the same service layer. A natural extension of stage B.
Database TDE integration	Testing against a vendor, not new code	Oracle, SQL Server and MongoDB already speak KMIP; the work is profile conformance and vendor testing, not new protocol.
Storage, backup and VM integrations	Testing against a vendor, not new code	As above, for VMware, NetApp and Veeam.
Secrets management	A different product surface	Static custody exists as SecretData. A Vault-style engine — dynamic credentials, leases, templating — is a different product surface.
Certificate lifecycle	A different product surface	Front the deployment with a real CA such as EJBCA. KMIP's Certify was never a CA and should not become one.
Algorithm breadth	Depends on the token	15 of 40 algorithms work on this token. A richer token activates more with no code change; the shim already probes.
Post-quantum (ML-KEM, ML-DSA, SLH-DSA)	Depends on the token	Needs a token that implements the algorithms and KMIP 3.0 to express them. SoftHSM's ML-DSA work is unreleased; vendor-range enums would interoperate with nothing.
KMIP 3.0	Schedulable, but only worth doing with a capable token	Enum and operation additions including Encapsulate and Decapsulate. Entirely in-protocol and schedulable, but only worth doing alongside a PQC-capable token.
KMIP interoperability certification	Depends on an external event	An OASIS interop event. The conformance tests are already mapped; this is process and scheduling.
PKCS#11 provider for applications	Build only if asked for	A library fronting the KMIP server, so applications that speak PKCS#11 plug in. Worth building only if asked for.
Tokenization / format-preserving encryption	Deliberately not this product	Neither a KMIP operation nor a token mechanism. A separate service.
FIPS 140-2/3 validated HSM	Buying and testing, not building	Belongs to the token. The PKCS#11 boundary makes the swap cheap; running the suite against validated hardware is the

		actual work.
Common Criteria / eIDAS	Buying and testing, not building	As above.

Six of these — the two cloud integrations, the two vendor validations, secrets management and certificate lifecycle — are ordinary engineering and would form a natural stage E once the REST layer exists to configure them. That would take the total closed to 34 of 42. The remaining eight depend on a supplier, a procurement decision, or an OASIS event, and no amount of planning moves them.

6 Risks worth naming now

Risk	Why it matters	Mitigation
A later endpoint bypasses the guard	All governance silently stops applying to that path, and nothing fails — the worst kind of defect this project has seen.	The reflection test in step 1, kept green forever. It is cheap and it is the single most valuable test in the plan.
Two authenticated front doors	The API doubles the authentication surface, and a leaked token is as good as a password.	Tokens are revocable, expiring, hashed at rest, and every issue and revoke is audited (step 3).
The audit write lock becomes the bottleneck	It already caps worker scaling at about 1.5×; REST traffic contends for the same lock.	Measure before and after step 11. If it binds, per-shard chains are the answer, and that is a design change, not a tuning exercise.
Tenancy migration surprises an existing deployment	Object names become unique per tenant rather than globally.	The step 12 migration places everything in a default tenant so behaviour is unchanged until a second tenant exists.
Scope creep into a security product	Tokenization, a CA and a secrets engine are all adjacent and all tempting.	They are named in section 5 as out of scope. Revisit deliberately, not by drift.
The plan outruns the verification environment	Steps 10 and 11 cannot be honestly completed without PostgreSQL, a second token and a registry.	Section 3 says so up front. A step whose gate cannot be demonstrated is not finished, whatever the code says.

One further note, from this project's own history: every serious defect found so far was invisible in the diff and obvious the moment something ran — an authentication bypass, plaintext in a WAL sidecar, a readiness probe that lied for eighteen seconds, a 25× latency cost, and a slide deck whose every diagram was missing. Each step's gate is written to be executed, not reviewed.